Labsheet 8 : BTCS301P-AS08

Computer Vision Lab

Khushi Choudhary CU240251564

In [4]:
```
import cv2
import numpy as np
from IPython.display import clear_output, display
import matplotlib.pyplot as plt

# Video Input Selection
video_source = "sample_video.mp4"
def show_frame(title, image):
    clear_output(wait=True)
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(10, 6))
    plt.imshow(image_rgb)
    plt.title(title)
    plt.axis("off")
    display(plt.gcf())
    plt.close()

# Open the video stream
cap = cv2.VideoCapture(video_source)

if not cap.isOpened():
    print(f"Error: Could not open video file '{video_source}'.")
    print("Please check if the file path is correct and the video file exists.")
else:
    # Lucas-Kanade Optical Flow (Sparse)
    feature_params = dict(maxCorners=100, qualityLevel=0.3, minDistance=7, blockSize=7)
    lk_params = dict(
        winSize=(15, 15),
        maxLevel=2,
        criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03),
    )
    color = np.random.randint(0, 255, (100, 3))

    ret, old_frame = cap.read()
    if not ret or old_frame is None:
        print("Error: Failed to read initial frame from video.")
    else:
        old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
        p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)
        mask = np.zeros_like(old_frame)
        frame_number = 0

        print("Processing Lucas-Kanade Sparse Optical Flow...")
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                print("End of video sequence reached (Sparse Flow).")
                break

            frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

            if p0 is not None and len(p0) > 0:
                p1, st, err = cv2.calcOpticalFlowPyrLK(
                    old_gray, frame_gray, p0, None, **lk_params
                )
            else:
                p1 = None
            if p1 is not None:
                good_new = p1[st == 1]
                good_old = p0[st == 1]
                for i, (new, old) in enumerate(zip(good_new, good_old)):
                    a, b = new.ravel()
                    c, d = old.ravel()
                    mask = cv2.line(
                        mask, (int(a), int(b)), (int(c), int(d)), color[i].tolist(), 2
                    )
                    frame = cv2.circle(
                        frame, (int(a), int(b)), 5, color[i].tolist(), -1
                    )
                img = cv2.add(frame, mask)
                if frame_number % 5 == 0:
                    show_frame("Lucas-Kanade Sparse Optical Flow", img)
                old_gray = frame_gray.copy()
                p0 = good_new.reshape(-1, 1, 2)
            else:
                p0 = cv2.goodFeaturesToTrack(frame_gray, mask=None, **feature_params)
            frame_number += 1
    cap.release()
```

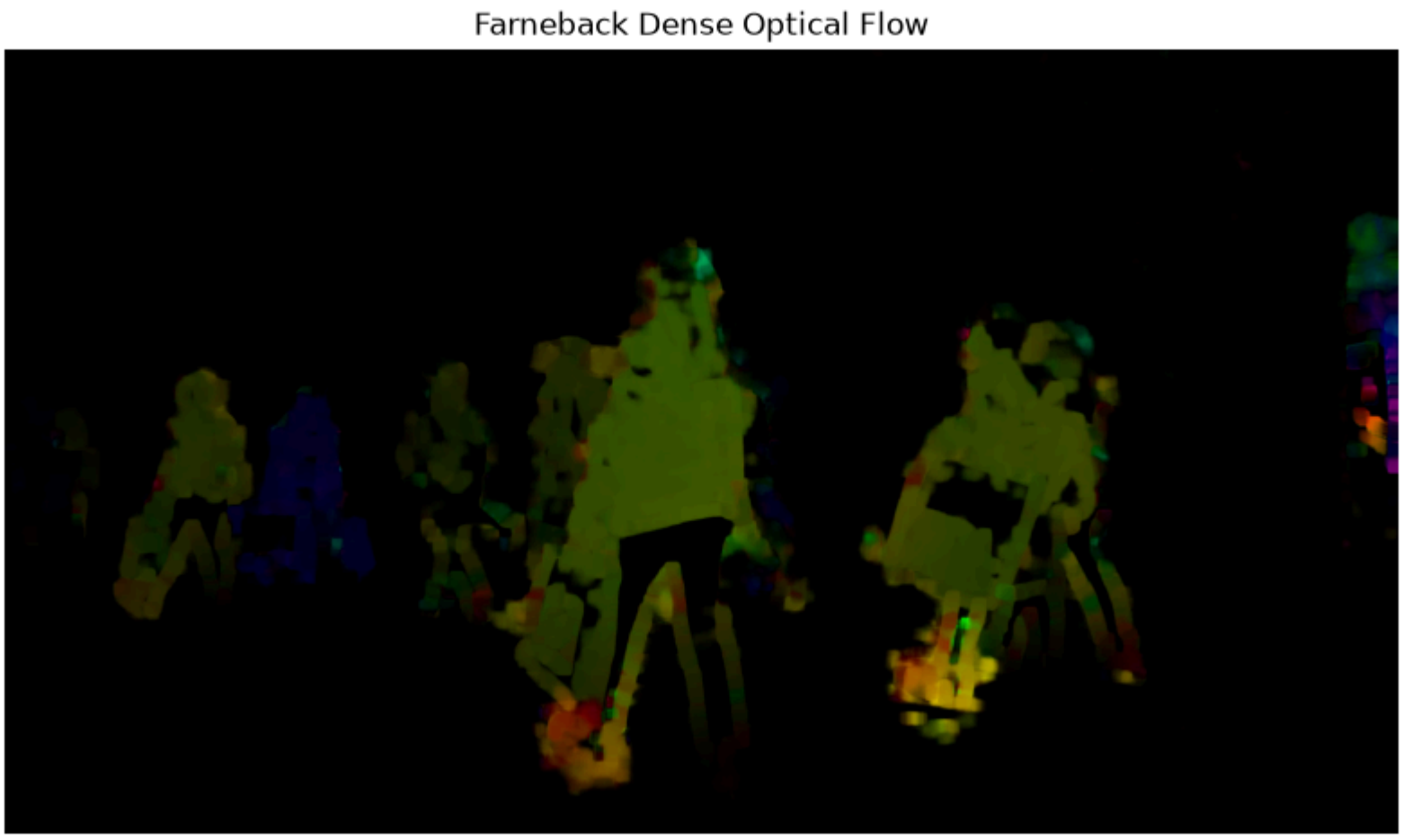
```python
# Farneback Optical Flow (Dense)
cap = cv2.VideoCapture(video_source)
ret, frame1 = cap.read()

if not ret or frame1 is None:
    print("Error: Could not read video for Farneback Dense Flow.")
else:
    prvs = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)
    hsv[..., 1] = 255
    frame_number = 0
    print("Processing Farneback Dense Optical Flow...")

    while cap.isOpened():
        ret, frame2 = cap.read()
        if not ret:
            print("End of video sequence reached (Dense Flow).")
            break
        next_frame = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
        flow = cv2.calcOpticalFlowFarneback(
            prvs, next_frame, None, 0.5, 3, 15, 3, 5, 1.2, 0
        )
        mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
        hsv[..., 0] = ang * 180 / np.pi / 2
        hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
        rgb = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

        if frame_number % 5 == 0:
            show_frame("Farneback Dense Optical Flow", rgb)
        prvs = next_frame
        frame_number += 1

cap.release()
print("Optical-flow processing completed.")
```




```
End of video sequence reached (Dense Flow).
Optical-flow processing completed.
```